Establishing Foundational Performance: Baseline Model Implementation and Evaluation

This notebook establishes the foundational performance benchmarks for the predictive task using a selection of industry-standard machine learning algorithms. The results from these models will serve as a crucial comparison point for the final proposed methodology.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
```

```
Mounted at /content/drive
```

Implementation of Baseline Model XGBoost

This entire process, from data loading to model evaluation, is consolidated into a single, comprehensive step to implement the XGBoost Classifier. The process begins by consolidating seven raw datasets—merging student demographics, course information, and time-series VLE and assessment logs into a unified structure. Feature engineering is performed by aggregating clicks and assessment scores per student to capture their engagement and commitment. The data is then cleaned by creating the binary is_dropout target variable, removing leaky identifiers like id_student, and applying One-Hot Encoding to categorical features. After handling missing values, the data is split (80/20, stratified) for training. The XGBoost Classifier is trained on this finalized data, and its performance is immediately evaluated on the test set using a Classification Report, Confusion Matrix, and the crucial ROC-AUC score to establish a high-performing baseline for the dissertation's primary research.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Loaded the 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    print("All datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Aggregate student VLE clicks
# Create a new DataFrame with aggregated clicks per student
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
```

```
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
# Merge with assessments table to get assessment metadata
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
# Create a new DataFrame with aggregated assessment scores per student
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Splitting ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns and other non-predictive columns.
# We are no longer using studentRegistration, so those columns are removed from the drop list.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
if 'date_x' in df.columns:
    columns_to_drop.append('date_x')
if 'date_y' in df.columns:
    columns_to_drop.append('date_y')
if 'absolute_date' in df.columns:
    columns_to_drop.append('absolute_date')

X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Handle categorical features with one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# 3.4. Handle potential missing values (if any remain)
X = X.fillna(X.mean())

# 3.5. Clean column names for XGBoost compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.col
X = X.rename(columns=cleaned_columns)

# 3.6. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nTraining data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

# --- Step 4: Model Training and Evaluation ---

# 4.1. Create an XGBoost model instance
xgb_model = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)

# 4.2. Train the model on the training data
print("\nTraining XGBoost model...")
xgb_model.fit(X_train, y_train)
print("XGBoost training complete.")

# 4.3. Make predictions on the test set
y_pred_xgb = xgb_model.predict(X_test)
y_pred_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]

# 4.4. Evaluate the model and print the results
print("\n--- XGBoost Model Evaluation ---")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb))

print("\nConfusion Matrix:")
```

```
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
print(cm_xgb)

# Visualize the confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_xgb, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Dropout', 'Dropout'], yticklabels=['Not Dropout', 'Dropout
plt.title('XGBoost Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Calculate ROC-AUC score
roc_auc_xgb = roc_auc_score(y_test, y_pred_proba_xgb)
print(f"\nROC-AUC Score: {roc_auc_xgb:.4f}")

# --- Step 5: Model Interpretation (Feature Importance) ---

# Get the feature importances from the model
feature_importances_xgb = xgb_model.feature_importances_
feature_names = X_train.columns

# Create a DataFrame for easy viewing and sorting
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances_xgb
})

# Sort by importance
importance_df = importance_df.sort_values(by='Importance', ascending=False)

print("\n--- Top 10 Most Important Features (XGBoost) ---")
print(importance_df.head(10))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All datasets loaded successfully.

Training data shape: (26074, 35)
Testing data shape: (6519, 35)

Training XGBoost model...
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:38:05] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost training complete.

--- XGBoost Model Evaluation ---

Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.93      0.90      3077
```

```
# --- Cross-Validation for XGBoost ---

# Create an XGBoost model instance
xgb_model = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)

# Define the cross-validation strategy
# n_splits=5 is a standard choice
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("\nPerforming 5-fold Cross-Validation...")

# Evaluate the model using different scoring metrics
accuracy_scores = cross_val_score(xgb_model, X, y, cv=cv, scoring='accuracy')
f1_scores = cross_val_score(xgb_model, X, y, cv=cv, scoring='f1')
roc_auc_scores = cross_val_score(xgb_model, X, y, cv=cv, scoring='roc_auc')

print("Cross-validation complete.")

# Print the results
print("\n--- Cross-Validation Results for XGBoost ---")
print(f"Average Accuracy: {np.mean(accuracy_scores):.4f} (+/- {np.std(accuracy_scores):.4f})")
print(f"Average F1-Score: {np.mean(f1_scores):.4f} (+/- {np.std(f1_scores):.4f})")
print(f"Average ROC-AUC: {np.mean(roc_auc_scores):.4f} (+/- {np.std(roc_auc_scores):.4f})")
```

Not Dropout Dropout

Predicted

```
ROC-AUC Score: 0.9623

--- Top 10 Most Important Features (XGBoost) ---
                           Feature  Importance
5          num_submitted_assessments    0.466577
4                        avg_score    0.043191
2                     total_clicks    0.037039
6                         gender_M    0.032769
0              num_of_prev_attempts    0.026929
20  highest_education_Lower_Than_A_Level    0.019399
1                  studied_credits    0.017039
31                 imd_band_90-100%    0.016986
12                  region_Scotland    0.016982
18           region_Yorkshire_Region    0.016930
```

```
Performing 5-fold Cross-Validation...
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:08] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:09] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.11/dist-packages/xgboost/training.py:183: UserWarning: [12:53:11] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

Implementation of Baseline Model Random Forest

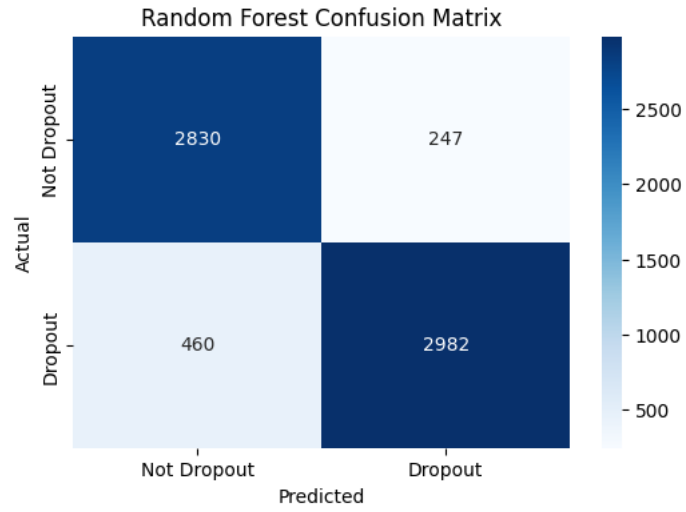
This entire process, from data loading to model evaluation, is consolidated into a single, comprehensive step to implement the Random Forest Classifier. This model is chosen as a robust, non-linear baseline because its ensemble approach (using multiple decision trees) is excellent at capturing complex feature interactions without extensive tuning, providing a strong initial ceiling for predictive performance. The preparation starts by consolidating all seven raw Open University datasets, merging demographic, VLE engagement, and assessment submission records onto the core student-course identifier. Key feature engineering steps include aggregating VLE clicks and calculating average assessment scores per student. The data is then refined by creating the binary is_dropout target variable, rigorously removing identifier and data-leakage columns, and using One-Hot Encoding for categorical variables. After filling any remaining missing values using the mean, the final feature set is split (80/20, stratified) to ensure an unbiased evaluation. The Random Forest Classifier is then trained, and its performance is assessed using a Classification Report, Confusion Matrix visualization, and the primary metric, the ROC-AUC score, concluding with an inspection of the Feature Importances to understand the most influential predictors of student dropout

```python
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()
# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
```

```
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Splitting ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns and other non-predictive columns.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')

X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Handle categorical features with one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# 3.6. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nTraining data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

# --- Step 4: Model Training and Evaluation ---

# 4.1. Create a Random Forest model instance
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)

# 4.2. Train the model on the training data
print("\nTraining Random Forest model...")
rf_model.fit(X_train, y_train)
print("Random Forest training complete.")

# 4.3. Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)
y_pred_proba_rf = rf_model.predict_proba(X_test)[:, 1]

# 4.4. Evaluate the model and print the results
print("\n--- Random Forest Model Evaluation ---")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

print("\nConfusion Matrix:")
cm_rf = confusion_matrix(y_test, y_pred_rf)
print(cm_rf)

# Visualize the confusion matrix
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Dropout', 'Dropout'], yticklabels=['Not Dropout', 'Dropout
plt.title('Random Forest Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Calculate ROC-AUC score
roc_auc_rf = roc_auc_score(y_test, y_pred_proba_rf)
print(f"\nROC-AUC Score: {roc_auc_rf:.4f}")

# --- Step 5: Model Interpretation (Feature Importance) ---

# Get the feature importances from the model
feature_importances_rf = rf_model.feature_importances_
feature_names = X_train.columns

# Create a DataFrame for easy viewing and sorting
importance_df_rf = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances_rf
})

# Sort by importance
importance_df_rf = importance_df_rf.sort_values(by='Importance', ascending=False)

print("\n--- Top 10 Most Important Features (Random Forest) ---")
print(importance_df_rf.head(10))
```

```
Mounted at /content/drive
All 7 datasets loaded successfully.

Training data shape: (26074, 35)
Testing data shape: (6519, 35)

Training Random Forest model...
Random Forest training complete.

--- Random Forest Model Evaluation ---

Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.92      0.89      3077
           1       0.92      0.87      0.89      3442

    accuracy                           0.89      6519
   macro avg       0.89      0.89      0.89      6519
weighted avg       0.89      0.89      0.89      6519


Confusion Matrix:
[[2830  247]
 [ 460 2982]]
```




```
ROC-AUC Score: 0.9573

--- Top 10 Most Important Features (Random Forest) ---
                                Feature  Importance
5              num_submitted_assessments    0.439729
4                              avg_score    0.282069
1                          studied_credits    0.035065
3                   num_vle_interactions    0.027813
2                            total_clicks    0.023219
6                               gender_M    0.014854
20  highest_education_Lower_Than_A_Level    0.013116
0                    num_of_prev_attempts    0.012614
32                         age_band_35_55    0.012447
19    highest_education_HE_Qualification    0.006989
```

```
# --- Cross-Validation for Random Forest ---

# Create a Random Forest model instance
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)

# Define the cross-validation strategy
# n_splits=5 is a standard choice
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("\nPerforming 5-fold Cross-Validation...")

# Evaluate the model using different scoring metrics
accuracy_scores = cross_val_score(rf_model, X, y, cv=cv, scoring='accuracy')
f1_scores = cross_val_score(rf_model, X, y, cv=cv, scoring='f1')
roc_auc_scores = cross_val_score(rf_model, X, y, cv=cv, scoring='roc_auc')

print("Cross-validation complete.")
```

```
# Print the results
print("\n--- Cross-Validation Results for Random Forest ---")
print(f"Average Accuracy: {np.mean(accuracy_scores):.4f} (+/- {np.std(accuracy_scores):.4f})")
print(f"Average F1-Score: {np.mean(f1_scores):.4f} (+/- {np.std(f1_scores):.4f})")
print(f"Average ROC-AUC: {np.mean(roc_auc_scores):.4f} (+/- {np.std(roc_auc_scores):.4f})")
```

```
Performing 5-fold Cross-Validation...
Cross-validation complete.

--- Cross-Validation Results for Random Forest ---
Average Accuracy: 0.8873 (+/- 0.0035)
Average F1-Score: 0.8895 (+/- 0.0038)
Average ROC-AUC: 0.9538 (+/- 0.0029)
```

Logistic Regression Baseline

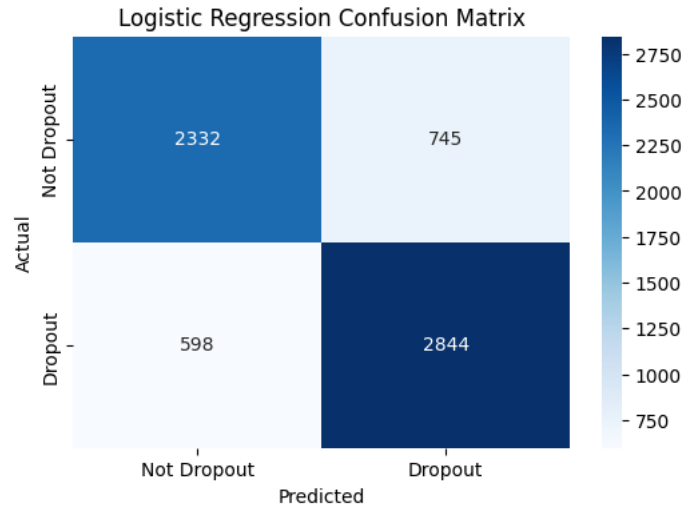
This comprehensive process details the implementation and evaluation of the Logistic Regression model, chosen as the fundamental linear baseline because its simplicity allows us to determine if the relationship between features and the target variable can be adequately captured without complex non-linear techniques. The process starts with consolidating seven Open University datasets, integrating demographic, VLE engagement, and assessment data and then performing feature engineering by aggregating clicks and average scores per student-course instance. Critical data preparation steps include converting the final_result column into the binary is_dropout target, rigorously removing columns that would cause data leakage (like individual registration/unregistration dates), and applying One-Hot Encoding to all categorical features. After imputation of minor missing values, the data is split (80/20, stratified) to ensure a fair evaluation. The Logistic Regression model is trained, and its performance is assessed using a Classification Report, a visual Confusion Matrix, and the ROC-AUC score. Finally, the model's coefficients are inspected to identify the most significant linear predictors of student dropout.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
```

```
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Splitting ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns and other non-predictive columns.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')

X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Handle categorical features with one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.cc
X = X.rename(columns=cleaned_columns)

# 3.6. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nTraining data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

# --- Step 4: Model Training and Evaluation ---

# 4.1. Create a Logistic Regression model instance
log_reg_model = LogisticRegression(solver='liblinear', random_state=42)

# 4.2. Train the model on the training data
print("\nTraining Logistic Regression model...")
log_reg_model.fit(X_train, y_train)
print("Logistic Regression training complete.")

# 4.3. Make predictions on the test set
y_pred = log_reg_model.predict(X_test)
y_pred_proba = log_reg_model.predict_proba(X_test)[:, 1]

# 4.4. Evaluate the model and print the results
print("\n--- Logistic Regression Model Evaluation ---")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Visualize the confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Dropout', 'Dropout'], yticklabels=['Not Dropout', 'Dropout'])
plt.title('Logistic Regression Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Calculate ROC-AUC score
roc_auc = roc_auc_score(y_test, y_pred_proba)
print(f"\nROC-AUC Score: {roc_auc:.4f}")
```

```
# --- Step 5: Model Interpretation (Feature Importance) ---

# Get the coefficients and feature names
coefficients = log_reg_model.coef_[0]
feature_names = X_train.columns

# Create a DataFrame to display the coefficients
coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients,
    'Absolute_Coefficient': np.abs(coefficients)
})

# Sort by absolute coefficient to see the most impactful features
coef_df = coef_df.sort_values(by='Absolute_Coefficient', ascending=False)

print("\n--- Top 10 Most Influential Features (Coefficients) ---")
print(coef_df.head(10))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.

Training data shape: (26074, 35)
Testing data shape: (6519, 35)

Training Logistic Regression model...
Logistic Regression training complete.

--- Logistic Regression Model Evaluation ---

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.76      0.78      3077
           1       0.79      0.83      0.81      3442

    accuracy                           0.79      6519
   macro avg       0.79      0.79      0.79      6519
weighted avg       0.79      0.79      0.79      6519


Confusion Matrix:
[[2332  745]
 [ 598 2844]]
```




```
ROC-AUC Score: 0.8682

--- Top 10 Most Influential Features (Coefficients) ---
                              Feature  Coefficient  Absolute_Coefficient
21     highest_education_No_Formal_quals     0.656215              0.656215
20  highest_education_Lower_Than_A_Level     0.555887              0.555887
8                         region_Ireland    -0.482800              0.482800
31                      imd_band_90_100%    -0.438959              0.438959
5              num_submitted_assessments    -0.410142              0.410142
28                      imd_band_60_70%    -0.396622              0.396622
0                   num_of_prev_attempts     0.339994              0.339994
30                      imd_band_80_90%    -0.308261              0.308261
29                      imd_band_70_80%    -0.296096              0.296096
27                      imd_band_50_60%    -0.279583              0.279583
```

```
# --- Cross-Validation for Logistic Regression ---
# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
import seaborn as sns


# Create a Logistic Regression model instance
log_reg_model = LogisticRegression(solver='liblinear', random_state=42)

# Define the cross-validation strategy
# n_splits=5 means the data will be split into 5 folds
# shuffle=True ensures the data is randomly shuffled
# random_state ensures reproducibility
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
print("\nPerforming 5-fold Cross-Validation...")

# Evaluate the model using different scoring metrics
accuracy_scores = cross_val_score(log_reg_model, X, y, cv=cv, scoring='accuracy')
f1_scores = cross_val_score(log_reg_model, X, y, cv=cv, scoring='f1')
roc_auc_scores = cross_val_score(log_reg_model, X, y, cv=cv, scoring='roc_auc')

print("Cross-validation complete.")

# Print the results
print("\n--- Cross-Validation Results for Logistic Regression ---")
print(f"Average Accuracy: {np.mean(accuracy_scores):.4f} (+/- {np.std(accuracy_scores):.4f})")
print(f"Average F1-Score: {np.mean(f1_scores):.4f} (+/- {np.std(f1_scores):.4f})")
print(f"Average ROC-AUC: {np.mean(roc_auc_scores):.4f} (+/- {np.std(roc_auc_scores):.4f})")

# A visual representation of cross-validation
```

```
Performing 5-fold Cross-Validation...
Cross-validation complete.

--- Cross-Validation Results for Logistic Regression ---
Average Accuracy: 0.7872 (+/- 0.0055)
Average F1-Score: 0.8042 (+/- 0.0058)
Average ROC-AUC: 0.8622 (+/- 0.0038)
```

MLP Baseline Implimentation

This comprehensive process details the implementation and evaluation of the Multi-Layer Perceptron (MLP) Classifier, chosen as the simple neural network baseline to evaluate the feasibility of a deep learning approach on this student data. The preparation begins by consolidating all seven raw Open University datasets and performing essential feature engineering by aggregating student VLE clicks and assessment scores to create features of engagement and performance. The data is then finalized by creating the binary is_dropout target, removing identifier and data-leakage columns, and using One-Hot Encoding for categorical variables. After minor imputation for missing data, the final feature set is split (80/20, stratified). Critically, because the MLP model is highly sensitive to the scale of input features, the data is subjected to Standard Scaling to ensure uniform feature magnitude. The MLP Classifier is then trained on this scaled data, and its performance is assessed on the hold-out test set using a Classification Report, a visual Confusion Matrix, and the robust ROC-AUC score to establish the final, non-traditional baseline benchmark.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---
```

```
# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Splitting ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns and other non-predictive columns.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')

X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Handle categorical features with one-hot encoding
X = pd.get_dummies(X, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.cc
X = X.rename(columns=cleaned_columns)

# 3.6. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# NEW STEP: Scale the data
# This is crucial for MLP models as they are sensitive to feature magnitudes.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTraining data shape: {X_train_scaled.shape}")
print(f"Testing data shape: {X_test_scaled.shape}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.

Training data shape: (26074, 35)
Testing data shape: (6519, 35)
```

```
# 2.1. Create an MLP model instance
# hidden_layer_sizes=(100,) means one hidden layer with 100 neurons.
# max_iter=500 is the number of epochs to train for.
mlp_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)

# 2.2. Train the model
print("\nTraining MLP model...")
mlp_model.fit(X_train_scaled, y_train)
```

```
print("MLP training complete.")

# 2.3. Make predictions
y_pred_mlp = mlp_model.predict(X_test_scaled)
y_pred_proba_mlp = mlp_model.predict_proba(X_test_scaled)[:, 1]

# 2.4. Evaluate the model
print("\n--- MLP Model Evaluation ---")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_mlp))

print("\nConfusion Matrix:")
cm_mlp = confusion_matrix(y_test, y_pred_mlp)
print(cm_mlp)

# Visualize the confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm_mlp, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Dropout', 'Dropout'], yticklabels=['Not Dropout', 'Drop
plt.title('MLP Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Calculate ROC-AUC score
roc_auc_mlp = roc_auc_score(y_test, y_pred_proba_mlp)
print(f"\nROC-AUC Score: {roc_auc_mlp:.4f}")
```

```
Training MLP model...
MLP training complete.
```